La cryptographie post-quantique

Bouali Abdallah

Numéro d'inscription : 10047

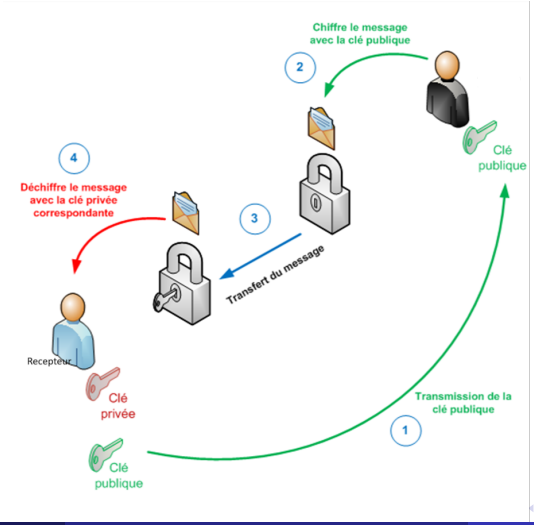
2024-2025

Plan :

- Introduction
- État des lieux des systèmes actuels
- Algorithme de Shor
- Les bases de la cryptographie post-quantique
- Comparaison et Conclusion

1 Introduction :

Principe :


Chiffre le message
avec la clé publique
2
Clé
publique
4
Déchiffre le message
avec la clé privée
correspondante
3
Transfert du message
Recepteur
Clé
privée
Transmission de la
clé publique
1
Clé
publique

2 État des lieux des systèmes actuels :

Étape 1 :

Génération des clés publique et privée

- Choisir deux nombres premiers p et q
- Calculer $n = p \times q$
- Calculer $\phi(n) = (p-1)(q-1)$
- Choisir e tel que $\text{pgcd}(e, \phi(n)) = 1$
- Calculer d tel que $e \times d \equiv 1 \pmod{\phi(n)}$

Chiffrement :
Pour chiffrer un message m :

$$c = m^e \bmod n$$

Où :

- c = message chiffré
- m = message en clair
- e = exposant public
- n = module public

Déchiffrement :
Pour déchiffrer un message c :

$$m = c^d \bmod n$$

Où :

- m = message déchiffré
- c = message chiffré
- d = exposant privé
- n = module public

Exemple d'utilisation :

Avec $P, Q = 157, 167$

S	a	i	n	t		R	e	m	i
83	97	105	110	116	32	82	101	109	105
156	77	153	83	48	114	15	35	140	153

Message en ASCII (ligne 2) et Message chiffré (ligne 3)

Table ASCII de référence :

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55		77	4D	115	M	109	6D	155	m

2 État des lieux des systèmes actuels :

Déchiffrement :

(n, e) : Clé publique $\qquad$ d : Clé privée

- Soit p, q deux nombres premiers tq $n = p \cdot q$
- $\varphi(n) = (p-1) \cdot (q-1)$
- e tq $e \wedge \varphi(n) = 1$
- d tq $d \times e \equiv 1[\varphi(n)]$ (calculé à l'aide de l'algorithme d'Euclide)

Il faut factoriser n

2 État des lieux des systèmes actuels :

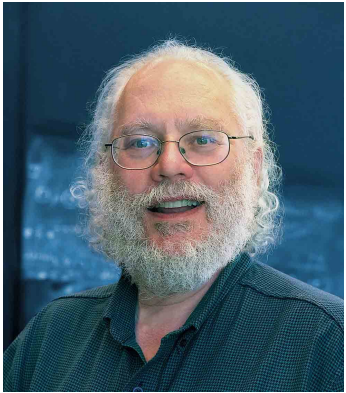
L'algorithme le plus rapide pour factoriser un nombre se fait en

$$O\left(e^{\sqrt{\ln(n)\ln(\ln(n))}}\right)$$

On suppose que l'ordinateur prend environ une microseconde à chaque opération :

Longueur	Nb d'opération	Durée en année
75	9×10^{12}	74
200	$1,2 \times 10^{23}$	$3,8 \times 10^{9}$
500	$1,3 \times 10^{39}$	$4,2 \times 10^{25}$

3 Algorithme de Shor :



Peter Shor
1998

- $0 < a < n$
- Si $\text{pgcd}(a, n) \neq 1$ alors $d = \text{pgcd}(a, N)$
- Si $\text{pgcd}(a, n) = 1$, alors $a \in (\mathbb{Z}/n\mathbb{Z})$
  donc $\exists k \in \mathbb{N}, a^k \equiv 1[n]$

3 Algorithme de Shor :

Hypothèse 1 : L'ordre r de a modulo n est pair.

$$a^r - 1 \equiv 0[n] \Leftrightarrow (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0[n]$$

Si r est impair, ou si $a^{r/2} \equiv -1[n]$ recommencer avec un autre a

Exemple :

Supposons que N=15, et on choisit a=2, Alors :

$$2^1 \equiv 2[15]$$
$$2^2 \equiv 4[15]$$
$$2^3 \equiv 8[15]$$
$$2^4 \equiv 1[15]$$

Et on a :

$$\text{pgcd}(3,15) = 3$$
$$\text{pgcd}(5,15) = 5$$

3 Algorithme de Shor :

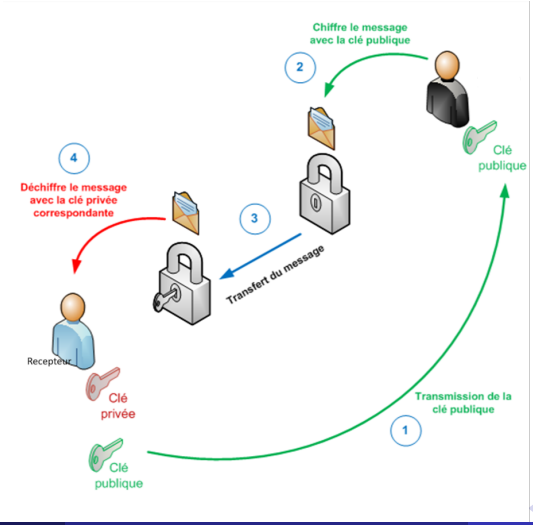
Impact sur la cryptographie :

- L'algorithme de Shor peut factoriser en temps polynomial
- Complexité : $O((\log n)^3)$ sur un ordinateur quantique
- Rend RSA, DSA, ECDSA vulnérables
- Nécessité de développer une cryptographie post-quantique

Menace quantique

Un ordinateur quantique suffisamment puissant pourrait casser RSA-2048 en quelques heures !

La méthode Kyber :



La méthode Kyber :

Basée sur :

- La théorie des réseaux euclidiens
- Un problème appelé LWE (Learning With Errors)

Problème LWE : Étant donné des échantillons $(a_i, b_i = \langle a_i, s \rangle + e_i)$, retrouver le vecteur secret s.

La méthode Kyber :

Principe :

Pour générer les clés :

1. *On prend* $A \in \mathbb{Z}_q^{k\times k}[x]/(x^n+1)$ et $S, e \in \mathbb{Z}_q^k[x]]/(x^n+1)$
2. *Calculer* $t = AS + e$

   Clé public $= (A, t)$, *et clé privé* $= (S, e)$

La méthode Kyber :

Principe :

Pour chiffrer :

1. *On prend* $r \in \mathbb{Z}_q^k[x]/(x^n+1)$ et $e_1 \in \mathbb{Z}_q^k[x]/(x^n+1)$ et $e_2 \in \mathbb{Z}_q[x]/(x^n+1)$
2. *Calculer* $u = A^T r + e_1$ et $v = t^T r + e_2 + \left\lfloor \frac{q}{2} \right\rfloor m$

   On envoie le message chiffré (u, v)

La méthode Kyber :

Principe :

Pour déchiffrer :

$$\textit{Il suffit de calculer } v - u^T s = \left\lfloor \frac{q}{2} \right\rfloor m + e_2 - e_1.s \ [q]$$

La méthode Kyber :

Pourquoi c'est sécurisé ?

$t = AS + e$ Clé public $= (A, t)$, et clé privé $= (S, e)$

Exemple :

$$\begin{pmatrix} 77 & 7 & 28 & 23 \\ 21 & 19 & 30 & 48 \\ 4 & 24 & 33 & 38 \\ 8 & 20 & 84 & 61 \end{pmatrix} \times \begin{pmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{pmatrix} = \begin{pmatrix} 2859 \\ 3508 \\ 3848 \\ 6225 \end{pmatrix} \pmod{q}$$

$$\begin{pmatrix} 77 & 7 & 28 & 23 \\ 21 & 19 & 30 & 48 \\ 4 & 24 & 33 & 38 \\ 8 & 20 & 84 & 61 \end{pmatrix} \times \begin{pmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{pmatrix} = \begin{pmatrix} 2859 \\ 3508 \\ 3848 \\ 6225 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \\ -2 \\ 2 \end{pmatrix} \pmod{q}$$

La méthode Kyber :
Exemple :

$q = 137, n = 4, k = 2$

$$A = \begin{pmatrix} 21 + 57x + 78x^2 + 43x^3 & 126 + 122x + 19x^2 + x^3 \\ 111 + 9x + 63x^2 + 33x^3 & 105 + 61x + 71x^2 + 64x^3 \end{pmatrix}$$

$$s = \begin{pmatrix} 1 + 2x - x^2 + 2x^3 \\ -x + 2x^3 \end{pmatrix}, e = \begin{pmatrix} 1 - x^2 + x^3 \\ -x + x^2 \end{pmatrix}$$

$$t = As + e = \begin{pmatrix} 55 + 96x + 123x^2 + 7x^3 \\ 32 + 27x + 127x^2 + 100x^3 \end{pmatrix}$$

La méthode Kyber :
Exemple :

On chiffre le message $m = 0111 \Leftrightarrow x + x^2 + x^3$

$$r = \begin{pmatrix} -2 + 2x + x^2 - x^3 \\ -1 + x + x^2 \end{pmatrix}, e_1 = \begin{pmatrix} 1 - 2x^2 + x^3 \\ -1 + 2x - 2x^2 + x^3 \end{pmatrix}, e_2 = 2 + 2x - x^2 + x^3$$

$$u = A^T r + e_1 = \begin{pmatrix} 56 + 32x + 77x^2 + 9x^3 \\ 45 + 21x + 2x^2 + 127x^3 \end{pmatrix}$$

et $v = t^T r + e_2 + 69m = 3 + 10x + 8x^2 + 123x^3$

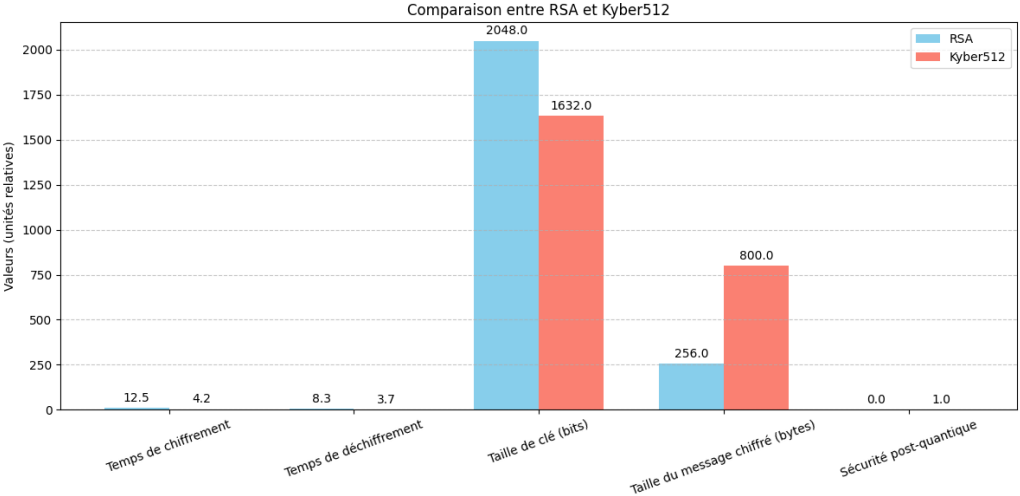
La méthode Kyber :
Exemple :

Pour déchiffrer :

$$v - s^T u = 4 + 60x + 79x^2 + 66x^3$$

et on calcule le reste de la division par $x^3 + 1$ *et on obtient* $x + x^2 + x^3$

5 Comparaison et Conclusion



Merci pour votre attention

Annexe

```python
def rsa(base, exposant, modulo):

    result = 1
    base = base % modulo
    while exposant > 0:
        if exposant % 2 == 1:
            result = (result * base) % modulo
        exposant = exposant // 2
        base = (base * base) % modulo
    return result

def inverse(a, m):

    t, new_t = 0, 1
    r, new_r = m, a

    while new_r != 0:
        quotient = r // new_r
        t, new_t = new_t, t - quotient * new_t
        r, new_r = new_r, r - quotient * new_r

    if r > 1:
        raise ValueError(f"{a} n'a pas d'inverse modulo {m}")
    if t < 0:
        t += m
    return t
```

Annexe

```python
from math import isqrt
from rsa import rsa, inverse

e = 65537
n = 2773  # n = 47 * 59

message = "Saint Remi"
ciphertext = [rsa(ord(c), e, n) for c in message]
print("Message chiffré :", ciphertext)

def factor(n):
    for i in range(2, isqrt(n) + 1):
        if n % i == 0:
            return i, n // i
    return None, None

p, q = factor(n)
if p is None:
    print("Échec de la factorisation")
    exit()

phi = (p - 1) * (q - 1)
d = inverse(e, phi)

decrypted = ''.join([chr(pow(c, d, n)) for c in ciphertext])
print("Message déchiffré :", decrypted)
```

```python
rom math import gcd

def pollards_rho(n):
    if n % 2 == 0:
        return 2  # Cas simple : divisible par 2

    x = 2
    y = 2
    c = 1
    f = lambda x: (x*x + c) % n

    while True:
        x = f(x)
        y = f(f(y))
        d = gcd(abs(x - y), n)
        if d == n:
            return None  # Échec
        if d > 1:
            return d
```

Annexe

```python
from random import random
from rsa import rsa, inverse

def is_pr(n, k=5):
    if n <= 3:
        return n == 2 or n == 3
    if n % 2 == 0:
        return False
    r, s = 0, n - 1
    while s % 2 == 0:
        r += 1
        s //= 2
    for _ in range(k):
        a = random. randrange(2, n - 1)
        x = rsa(a, s, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = rsa(x, 2, n)
            if x == n - 1:
                break
            else:
                return False
        return True


def gen_pr(bits):
    while True:
        n = random.getrandbits(bits)
        if n%2 == 0:
            n=1
        if is_pr(n):
            return n
```

```python
def gen_cle(p,q):
    n= p*q
    phi= (p-1)*(q-1)
    e = 65537
    d = inverse(e,phi)

def chifre(msg, cle_pc):
    e, n = cle_pc
    msg_chifre = [rsa(ord(char),e ,n) for char in msg]
    return msg_chifre

def dechifre(msg_chifre, cle_pv):
    d, n = cle_pv
    msg_dec -[chr(rsa(char, d, n)) for char in msg_chifre]
    return ''.join(msg_dec)

msg = 'Saint Remi'

print(chifre(msg, (157,167)))
```

Annexe

```python
from qiskit import QuantumCircuit
from math import gcd

def controlled_mod_exp(a, N, power):
    qc = QuantumCircuit(4)
    qc.barrier()
    return qc

def qpe_shor_circuit(a, N, n_count=8):
    qc = QuantumCircuit(n_count + 4, n_count)

    for q in range(n_count):
        qc.h(q)

    for q in range(n_count):
        qc.append(controlled_mod_exp(a, N, 2**q), [q] + [n_count + i for i in range(4)])

    qc.barrier()

    for i in range(n_count // 2):
        qc.swap(i, n_count - i - 1)
    for j in range(n_count):
        for m in range(j):
            qc.cp(-3.14159 / float(2 ** (j - m)), m, j)
        qc.h(j)

    qc.measure(range(n_count), range(n_count))
    return qc
```

Annexe

```python
import random

q = 3329
n = 8

def gen_poly():
    return [random.randint(0, q - 1) for _ in range(n)]

def encode(message_bits):
    poly = [(q // 2) if bit == 1 else 0 for bit in message_bits]
    while len(poly) < n:
        poly.append(0)
    return poly

def decode(poly):
    return [1 if coeff > q // 4 else 0 for coeff in poly]

def add_noise(poly, noise_level=5):
    return [(coeff + random.randint(-noise_level, noise_level)) % q for coeff in poly]

def keygen():
    A = gen_poly()
    s = gen_poly()
    return A, s

# Simulation
message = [1, 0, 1, 1, 0, 1]
A, s = keygen()
```

Annexe

```python
import matplotlib.pyplot as plt
import numpy as np

# Données comparatives
x = ["Temps de chiffrement", "Temps de déchiffrement", "Taille de clé (bits)", "Taille du message chiffré (bytes)", "Sécurité post-quantique"]
rsa = [12.5, 8.3, 2048, 256, 0]  # RSA
kyber = [4.2, 3.7, 1632, 800, 1]  # Kyber512

x_pos = np.arange(len(x))
width = 0.35

fig, ax = plt.subplots(figsize=(12, 6))
bars1 = ax.bar(x_pos - width/2, rsa, width, label="RSA", color='skyblue')
bars2 = ax.bar(x_pos + width/2, kyber, width, label="Kyber512", color='salmon')

ax.set_ylabel("Valeurs (unités relatives)")
ax.set_title("Comparaison entre RSA et Kyber512")
ax.set_xticks(x_pos)
ax.set_xticklabels(x, rotation=20)
ax.legend()

for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.1f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom')

plt.tight_layout()
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```